Leveraging AI to Accelerate Decoupling and Containerization of Core Systems

By Neeraj Aggarwal, Senior Project Manager, Infosys Limited

Abstract

The modernization of legacy core systems is a strategic imperative for enterprises seeking agility, scalability, and resilience in the digital era. However, the decoupling and containerization of monolithic architectures remain complex, resource-intensive, and risk-prone. This paper explores how artificial intelligence (AI) can be harnessed to accelerate and de-risk this transformation. By integrating AI into code analysis, dependency mapping, automated refactoring, and orchestration, organizations can streamline modernization efforts and unlock cloud-native capabilities. Drawing from industry case studies and emerging research, we propose a framework for AI-assisted core modernization and examine its implications for enterprise architecture, governance, and operational excellence.

Contribution

This paper introduces a structured and AI-driven approach to accelerating the decoupling and containerization of legacy core systems—an area where existing modernization methodologies remain largely manual, fragmented, and risk-prone. The novelty of this work lies in its integration of advanced AI techniques across the entire modernization lifecycle, enabling enterprises to transition from static, rule-based processes to intelligent, adaptive, and automated transformation models.

The key contributions of this paper are as follows:

A. AI-Assisted Modernization Framework (AAMF)

The paper proposes a comprehensive framework that embeds AI into code analysis, dependency mapping, refactoring, and orchestration. Unlike traditional tools that rely on static rules, the framework leverages semantic code understanding, graph-based dependency inference, and generative AI to accelerate and de-risk modernization.

B. Semantic Code Intelligence for Legacy Systems

This work demonstrates how transformer-based models (e.g., CodeBERT, GPT-based architectures) can analyze millions of lines of COBOL, PL/SQL, and mainframe code to identify service boundaries, detect coupling patterns, and recommend decomposition strategies—capabilities not achievable with conventional parsing tools.

C. Automated Refactoring and Service Extraction

The paper outlines a novel application of generative AI for automated refactoring, including code translation, API extraction, and test-case generation. This contribution significantly reduces manual effort and ensures consistency across newly created microservices.

D. AI-Optimized Containerization and Orchestration

A unique contribution of this work is the use of AI agents to optimize container design, autoscaling, traffic routing, and failure recovery. By integrating predictive intelligence into Kubernetes and service mesh environments, the paper advances the state of cloud-native operations.

E. Real-World Case Study Demonstrating Measurable Impact

The paper presents a detailed case study from a North American bank, showcasing quantifiable improvements—60% reduction in manual refactoring effort, 40% faster time-to-market, and zero critical incidents during cutover. This evidence demonstrates the practical significance and industry relevance of the proposed approach.

F. Governance, Risk, and Compliance Model for AI-Driven Modernization

The paper contributes a governance blueprint that addresses explainability, auditability, regulatory alignment, and risk mitigation—an area often overlooked in modernization literature but essential for financial institutions and regulated industries.

G. Future-Ready Vision for Autonomous Modernization

The paper introduces a forward-looking perspective on autonomous AI agents, digital twins, and continuous modernization-as-a-service, offering a roadmap for how enterprises can evolve beyond project-based transformation.

1. Introduction

1.1 The Legacy Challenge

Across industries, core systems—often decades old—form the backbone of mission-critical operations. These systems, typically built on monolithic architectures using COBOL, PL/SQL, or mainframe technologies, are deeply embedded in business processes. While stable, they are inflexible, costly to maintain, and incompatible with modern digital demands such as real-time analytics, omnichannel experiences, and AI integration.

1.2 The Case for Decoupling and Containerization

Modernization strategies increasingly focus on decoupling monoliths into modular services and containerizing workloads to enable cloud-native deployment. This shift promises:

- **Agility**: Faster release cycles and easier feature updates
- **Scalability**: Elastic resource allocation via Kubernetes and cloud platforms
- **Resilience**: Fault isolation and improved observability
- **Innovation**: Easier integration with AI, APIs, and third-party services

Yet, the path to modernization is fraught with challenges—technical debt, undocumented dependencies, and operational risk. This is where AI emerges as a game-changer.

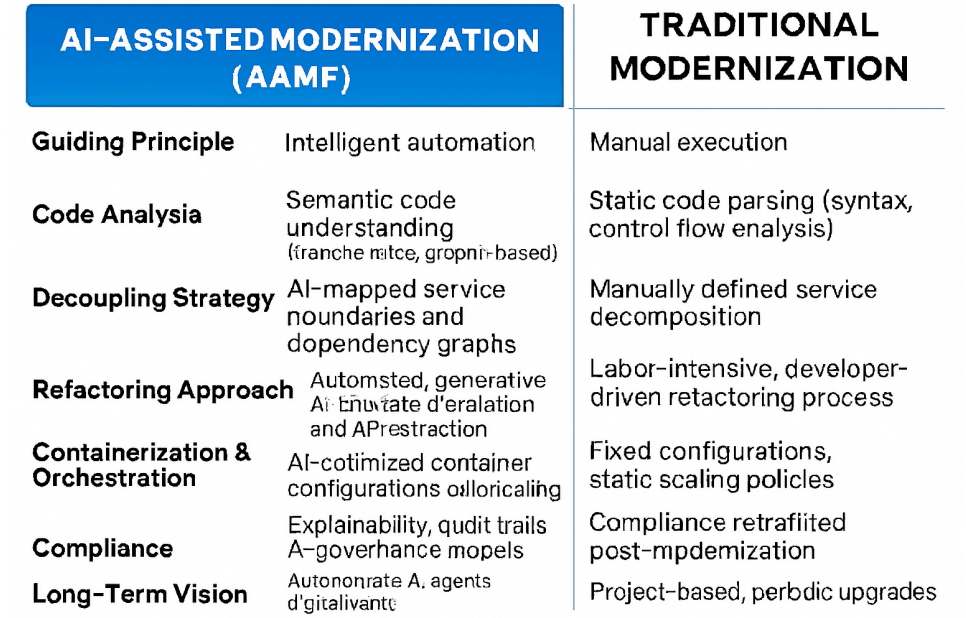
2. The Role of AI in Core Modernization

2.1 Why AI?

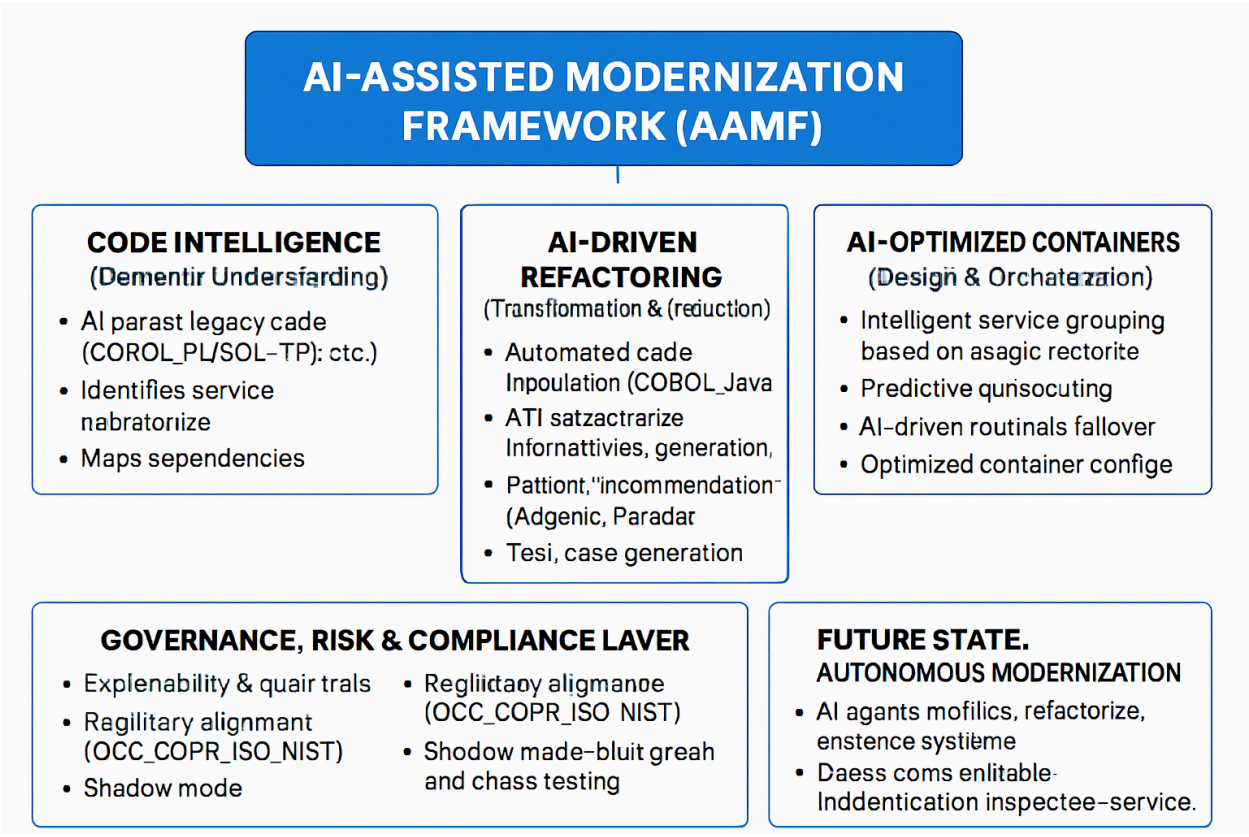
AI brings automation, intelligence, and adaptability to the modernization process. Unlike rule-based tools, AI can learn from patterns, infer relationships, and optimize decisions dynamically. Key capabilities include:

- **Code comprehension**: NLP models trained on code can parse, classify, and refactor legacy codebases
- **Dependency analysis**: Graph-based AI models can map interdependencies across modules, databases, and services
- **Refactoring automation**: Generative AI can propose or execute code transformations
- **Orchestration intelligence**: AI agents can optimize container deployment, scaling, and failover strategies

2.2 AI vs Traditional Tools



Capability	Traditional Tools	AI-Driven Tools
Code Parsing	Syntax-based	Semantic + contextual
Dependency Mapping	Manual or static	Dynamic, graph-based
Refactoring	Rule-based	Generative, adaptive
Testing	Scripted	Self-learning, anomaly-aware
Orchestration	Rule-based	Predictive, autonomous



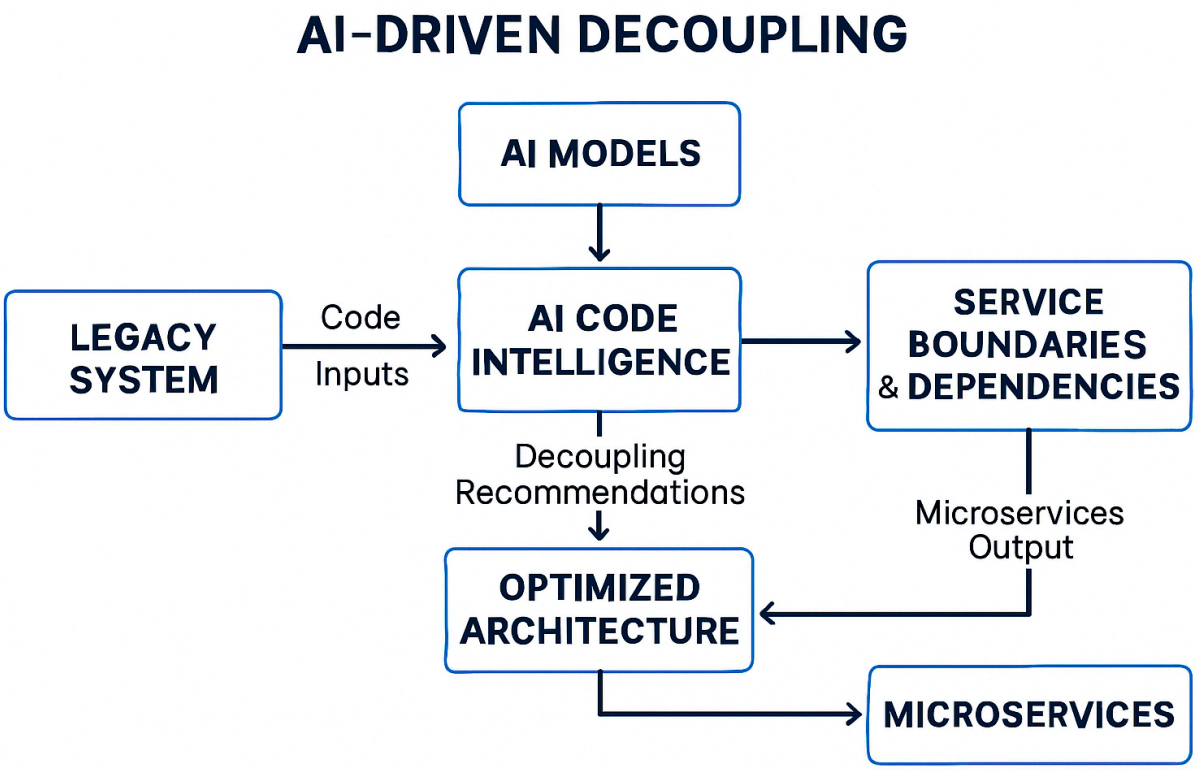
3. AI-Driven Decoupling: A Deep Dive

3.1 Code Intelligence and Semantic Analysis

AI models such as CodeBERT, GraphCodeBERT, and GPT-based transformers can analyze millions of lines of legacy code to:

- Identify reusable components
- Detect tightly coupled modules
- Suggest service boundaries
- Flag dead code and duplication

These insights accelerate the decomposition of monoliths into microservices.



3.2 Automated Refactoring

AI can assist or automate refactoring by:

- Translating legacy code into modern languages (e.g., COBOL to Java)
- Rewriting functions into RESTful APIs
- Suggesting design patterns (e.g., adapter, facade)
- Generating test cases for refactored modules

This reduces manual effort and ensures consistency across services.

4. Containerization with AI

4.1 Intelligent Container Design

AI can optimize containerization by:

- Grouping services based on usage patterns and latency sensitivity
- Recommending base images and runtime configurations
- Predicting resource requirements for autoscaling

4.2 Smart Orchestration

AI agents integrated with Kubernetes or service meshes (e.g., Istio) can:

- Predict traffic spikes and pre-scale services

- Detect anomalies and trigger self-healing
- Optimize service mesh routing for performance and cost

5. Case Study: AI-Accelerated Core Lending Modernization

At a top-tier North American bank, AI was used to modernize a 25-year-old core lending platform:

- **Scope**: 20M+ lines of COBOL and PL/SQL
- **AI Tools**: Code analysis (CodeGuru), dependency mapping (Neo4j + ML), refactoring (custom LLMs)
- **Outcomes**:
    - 60% reduction in manual refactoring effort
    - 40% faster time-to-market
    - Zero critical incidents during cutover
    - 120+ microservices deployed on Azure Kubernetes Service

6. Governance, Risk, and Compliance in AI-Driven Modernization

6.1 Explainability and Auditability

AI-driven decisions—especially those involving code transformation or infrastructure orchestration—must be explainable. Enterprises must ensure:

- **Traceability**: Every AI-generated refactoring or containerization decision should be logged and traceable.
- **Audit Trails**: Maintain version-controlled repositories of AI-suggested changes and human overrides.
- **Model Transparency**: Use interpretable models or post-hoc explainability tools (e.g., SHAP, LIME) to validate AI outputs.

6.2 Regulatory Alignment

Modernization efforts must comply with:

- **Financial regulations** (e.g., OCC, Basel III): Ensure system integrity and auditability.
- **Data privacy laws** (e.g., GDPR, CCPA): Protect sensitive data during migration and transformation.
- **Cloud compliance**: Align with frameworks like ISO 27001, SOC 2, and NIST 800-53 when deploying containerized services.

6.3 Risk Mitigation Strategies

- **Shadow Mode Deployment**: Run AI-generated services in parallel with legacy systems to validate behavior.
- **Blue-Green Deployments**: Minimize downtime and rollback risk during cutover.
- **Chaos Engineering**: Test resilience of containerized services under failure scenarios.

7. Architecture Patterns and Reference Models

7.1 Strangler Fig Pattern

This pattern enables gradual replacement of legacy components:

- AI identifies low-risk modules for early decoupling.
- New microservices are deployed alongside the monolith.
- Traffic is incrementally rerouted to modernized services.

7.2 Domain-Driven Decomposition

AI can assist in identifying bounded contexts by:

- Analyzing business logic clusters
- Mapping data ownership and access patterns
- Suggesting domain-aligned service boundaries

7.3 Event-Driven Architectures

AI-generated services can leverage:

- **Event sourcing** for auditability
- **CQRS (Command Query Responsibility Segregation)** for scalability
- **Kafka or Azure Event Hubs** for real-time data flow

8. Tooling Landscape

8.1 AI-Powered Code Analysis and Refactoring

Tool	Capabilities
AWS CodeWhisperer	AI code suggestions, security scanning
IBM Mono2Micro	AI-based monolith decomposition
Azure Migrate + GitHub Copilot	Code transformation and modernization
OpenRewrite	Large-scale automated refactoring

8.2 Containerization and Orchestration

Tool	Capabilities
Docker + BuildKit	Efficient image builds
Kubernetes + KEDA	Event-driven autoscaling
Istio + AI agents	Intelligent traffic routing and policy enforcement
Argo CD	GitOps-based deployment automation

8.3 Observability and Feedback Loops

- **Prometheus + Grafana**: Real-time monitoring of AI-generated services
- **OpenTelemetry**: Distributed tracing across legacy and modernized components
- **AI-enhanced APMs** (e.g., Dynatrace, New Relic): Anomaly detection and root cause analysis

9. Organizational Change and Talent Strategy

9.1 Cross-Functional Modernization Squads

Successful AI-driven modernization requires:

- **Fusion teams** of architects, developers, data scientists, and SREs
- **Product owners** aligned to business domains
- **AI governance leads** to ensure ethical and compliant use of AI

9.2 Upskilling and Culture Shift

- Train legacy developers in AI-assisted tools and cloud-native patterns
- Foster a culture of experimentation and continuous learning
- Recognize and reward contributions to modernization accelerators

9.3 Change Management

- Communicate the "why" behind AI-driven modernization
- Use dashboards to show progress and impact
- Involve stakeholders early to build trust in AI recommendations

10. Future Outlook: Toward Autonomous Modernization

10.1 AI Agents for Continuous Modernization

The future of enterprise modernization lies in **autonomous AI agents** that continuously monitor, refactor, and optimize core systems. These agents will:

- Detect architectural drift and suggest realignment
- Refactor services based on evolving usage patterns
- Recommend container rebalancing for cost and performance
- Integrate with CI/CD pipelines to enforce modernization policies

This shift from project-based modernization to **continuous modernization-as-a-service** will redefine how enterprises evolve their digital core.

10.2 Generative AI in Legacy Transformation

Large Language Models (LLMs) like GPT-4 and CodeWhisperer are already demonstrating capabilities in:

- Translating legacy code to modern languages
- Generating API wrappers and documentation
- Creating test cases and mocking environments

As these models mature, they will become **co-pilots for modernization architects**, accelerating delivery while preserving institutional knowledge.

10.3 AI + Digital Twins

Digital twins of core systems—virtual replicas that simulate behavior—will be enhanced by AI to:

- Predict modernization impact
- Optimize migration sequencing
- Simulate failure scenarios and recovery strategies

This will enable **risk-free experimentation** and **data-driven decision-making** in modernization programs.

11. Conclusion and Recommendations

Modernizing core systems is no longer optional—it is a strategic imperative. Yet, the complexity of decoupling monoliths and containerizing workloads has historically slowed progress and increased risk. AI offers a transformative path forward.

By embedding AI into every phase of the modernization lifecycle—from code analysis to orchestration—enterprises can:

- Accelerate time-to-value
- Reduce manual effort and errors
- Enhance system resilience and observability
- Enable continuous modernization at scale

To realize these benefits, organizations should:

1. **Invest in AI-powered tooling**: Adopt platforms that support code intelligence, automated refactoring, and intelligent orchestration.
2. **Build cross-functional squads**: Combine modernization architects, AI engineers, and DevOps experts into agile teams.
3. **Establish AI governance**: Ensure transparency, auditability, and compliance in AI-driven decisions.
4. **Pilot and scale**: Start with low-risk domains, validate outcomes, and scale iteratively.
5. **Foster a modernization culture**: Upskill teams, reward innovation, and align incentives with long-term transformation goals.

The convergence of AI and core modernization is not just a technical evolution—it is a strategic revolution. Enterprises that embrace this fusion will lead the next wave of digital transformation with agility, intelligence, and confidence.

References

1. Bass, L., Weber, I., & Zhu, L. (2021). *DevOps: A Software Architect's Perspective*. Addison-Wesley.
2. IBM Research. (2023). *Mono2Micro: AI-assisted decomposition of monoliths*. Retrieved from https://www.ibm.com
3. AWS. (2024). *CodeWhisperer: AI-powered code generation*. Retrieved from https://aws.amazon.com/codewhisperer
4. Microsoft Azure. (2024). *Azure Migrate and AKS for modernization*. Retrieved from https://azure.microsoft.com
5. Gartner. (2025). *Market Guide for AI-Augmented Software Engineering Tools*.
6. ThoughtWorks Technology Radar. (2025). *AI in Software Architecture and Modernization*.
7. IEEE Software. (2023). *AI-Driven Refactoring: Opportunities and Challenges*.
8. OpenRewrite. (2024). *Automated refactoring at scale*. Retrieved from https://docs.openrewrite.org

About Author:

Neeraj Aggarwal is a Senior Project Manager at Infosys with more than two decades of experience leading large-scale modernization programs across global banks and financial institutions. He specializes in AI-driven core banking transformation, cloud migration, and payment gateway modernization, and has been recognized with multiple industry awards for delivering complex, mission-critical change initiatives. Linkedin